```
  clusters, noise, mergeList ← mapReduce(mappers, reducers)
                  #5
  return combine(clusters, noise,
mergeList)                                    #6
```

#1 Method `MRdbscan` takes a list of points, the number of desired cells in which the dataset should be sharded, and the parameters for DBSCAN algorithm, the radius and minimum number of points that define a dense zone.

#2 Break down the dataset into `numShards` regular cells; each ell will be extended for a length `eps` in all directions to define a shards. Since we need grid-based sharding, we do need a customized sharding function. This method also needs to return the list of adjacent shards (better if a list of pairs of adjacent shards, without duplicates), and the list of points in the margin regions for each pair of adjacent shards.

#3 Initialize `numShards` mappers, each will run DBSCAN clustering method and will hold a copy of one of the dataset's shards. The mappers will be alive (and hold the same copy of the input) for the whole lifetime of this method.

#4 Initialize one reducer for each pair of shards in the adjacency list. The reducers will run the `mergeClusters` method and will be alive for the whole method's lifetime, and they will hold data about the adjacent shards they operate on, and the margin points between those two shards.

#5 Run MapReduce, using the mappers and reducers already created. Mappers will use the shard of data they already hold as input. Reducers will read their input (clusters and outliers local to shards) from the mappers (plus, they